

Pandemaniac Report

Riti Agarwal, Purvi Sehgal

February 2024

1 Introduction

This is our final report for the pandemaniac project. It explains our thought process and logic behind the strategies we chose to implement, along with why we thought they would be useful. In the project, we were able to beat ta baseline and ta target, but we ties dead even with ta hard. We go to the second round of the tournament.

2 Strategy 1 - thompsons_sampling_v2() (UCB - 1)

We chose to implement a version of the UCB-1 algorithm. This algorithm is part of the multi-arm bandit suite of algorithms, which aim to explore and exploit at the same time. In the context of our graph, balancing exploration and exploitation refers to making decisions that strike a trade-off between two conflicting objectives:

- Exploration:
 - The algorithm needs to explore different options to gather information about their potential rewards.
 - By exploring, the algorithm can discover nodes that may have higher rewards but have not been frequently selected in the past.
 - Exploration is crucial for gaining a better understanding of the environment and avoiding premature convergence to suboptimal choices.
- Exploitation:
 - The algorithm should exploit the knowledge it has accumulated so far by favoring nodes with higher estimated rewards.
 - Exploitation involves choosing actions that are believed to have higher immediate rewards based on the current knowledge.
 - Exploiting known high-reward options is essential for maximizing the cumulative reward over time.

The UCB algorithm achieves this balance by incorporating an exploration term in the selection process.

$$\text{mu_bar}[i] = \max \left(\text{self.means}[i] + \sqrt{\frac{2 \cdot \log(\text{self.iter})}{\text{self.times}[i]}}, 2 \right)$$

Here, `self.means[i]` represents the estimated mean reward for the *i*th node. `self.times[i]` is the count of how many times the *i*th node has been selected. `np.sqrt(2 * np.log(self.iter)/(self.times[i]))` is the exploration term, which increases with the logarithm of the total number of iterations `self.iter` and decreases with the number of times the node has been selected. The `np.maximum(..., 2)` ensures that the exploration term is not dominant, and it introduces a lower bound to prevent overly aggressive exploration.

By using this exploration term in the calculation of `mu_bar`, the algorithm ensures that less-explored nodes have higher estimates and are more likely to be selected. As more information is gathered (more iterations), the exploration term diminishes, and the algorithm tends to exploit nodes with higher estimated rewards.

We first set our reward function to be whether or not we beat max degree. In the `run_simulation` code provided to us, we went back and changed the code to also return how many nodes each was conquered by each specific seed node. So, this became our reward function. The number of nodes conquered by each seed node against the max degree strategy was our reward function, so in our algorithm, we were choosing the nodes that beat max_degree by the most margin.

As expected, since this algorithm is programmed to beat max degree, it beat max degree every time. But, this was losing to some of the centrality measures, and since we did not know if we had beaten a baseline on the first day, we wanted to improve this algorithm for the next day.

An important factor to note was that since we were doing machine learning, we did not really have to think about what seed nodes other team members would choose. Since we felt this was a very different strategy, we did not worry about choosing the same nodes as another team.

In our github file, this function is called `thompsons_sampling_v2()`. Also, this is in file `Strategy1.2.3.4`. It also exists in file `Strategy1.2.3.4_diff_params`. `Strategy1.2.3.4_diff_params` file contains the same overarching strategy with more parameter changes.

3 Strategy 2 - `thompsons_sampling_new()` Thompson Sampling 2

In this strategy, we made our reward function a weighted sum of degrees at three levels. The reasoning for this strategy was that we want to ideally choose nodes that not only have a high number of first degree neighbors, but also a high number of second and third degree of neighbors because that is what

is going to affect spread in subsequent time periods. We made sure to not include nodes that had already been counted to account for acyclic behavior. We wrote a function that takes in the starting node and uses breadth first search to calculate the total number of nodes at each of the levels. It returns a list of size three (representing each of the three levels), where the 0th element represents the number of first degree neighbors, the 1st element represents the number of second degree neighbors (aka neighbors of the first-degree neighbors), etc. We stopped it at three levels because it seemed like a majority of nodes in the graph were starting to get included, which we didn't want. Then, once we returned a list with the number of nodes at each level, we assigned a weight to each level, denoting it's significance. We created a list with weights because the number of nodes in the first degree should matter more than the number of nodes in the second degree which should matter more than the number of nodes in the third degree. We then calculated the weighted sum of the list items and then normalized the sum, since if we wanted to add centrality measures, it would give equal importance to each of the measures. During the testing phase, we adjusted the weighting until we starting winning on a larger percentage of the graphs. The final weighting we used was around [.6, .25, .15], and it was chosen through trial and error.

Here, we used the same UCB algorithm as above, except this time we used a different reward function which we were hoping would beat different centrality measures. So, keeping our thompsons sampling algorithm the same, we implemented it with a new reward function.

Also, this is in file Strategy1_2_3_4. It also exists in file Strategy1_2_3_4.diff_params. Strategy1_2_3_4.diff_params file contains the same overarching strategy with more parameter changes.

4 Strategy 3 - thompsons_sampling () thomsp-son sampling

Initialization:

- `alpha_0` and `beta_0` are set to 1 for a uniform prior.
- `self.iter` is initialized to 0 to keep track of the number of iterations.
- `self.means` is initialized with the normalized degrees of nodes in the graph.
- `self.times` is initialized as an array of zeros to track the number of times each node is picked.
- `sample_values` is initialized as an array of zeros to store sample values from the Beta distribution.

Bayesian Update Loop:

- For each node (`i`) in the range `self.num_nodes`:
 - * Update the parameters of the Beta distribution (`alpha` and `beta`) using Bayesian updating.
 - * Sample a value from the Beta distribution.

- * Select the top `self.num_seeds` nodes based on the sampled values.
- * Convert the selected nodes to strings and calculate scores based on reward function used in the previous algorithm.
- * Update the count of times each selected node has been picked.
- * Update the mean rewards using a formula similar to the one in the UCB algorithm.

Final Selection:

- After the loop, select the top `self.num_seeds` nodes based on the updated mean rewards.
- Return the selected nodes.

Here, the code utilizes a Bayesian approach with the Beta-Binomial distribution to model the uncertainty and update beliefs about the rewards associated with each node in the graph. The selection of nodes is influenced by both the observed rewards and the prior distribution, allowing the algorithm to adapt its choices based on new information over iterations. The final selection is based on the updated mean rewards.

This strategy lost to our previously described approach almost every time, and so we chose not to run this. We were curious to see if this would work, but perhaps we did not spend enough time thinking about how the updates could work better, we felt it was more important for us to fine tune our other strategies.

5 strategy 4 - competing_strategy() Combine Thompson Sampling and Max Degree

The algorithm combines the strengths of Thompson Sampling and Max Degree, leveraging the betweenness centrality metric to prioritize nodes for seeding. The approach aims to capture the most influential nodes by considering both exploration-exploitation aspects (Thompson Sampling) and local neighborhood influence (Max Degree). Here, our strategy was to get the top k nodes selected by Thompson's sampling, and get the top k nodes selected by max degree. So, we then have a list of 2k nodes. To pick from these 2k nodes, we have 2 strategies. If the graph cannot be broken up into clusters, we ordered the 2k nodes by their highest betweenness values, and take the top k nodes. If the graph can be broken up into clusters, the algorithm identifies and sorts clusters in the graph based on size in descending order (sorted_clusters). The top 4 largest clusters are selected (largest_clusters). Next, the algorithm iterates through each node in the combined set of 2k nodes c(combined_nodes). For each combined node, it checks if the node is present in any of the largest clusters. If a node is found in a cluster, it is considered important and added to the list of important nodes (imp_nodes). This process ensures that only nodes belonging to the largest clusters are considered as potentially influential. The algorithm then sorts the

`imp_nodes` by betweenness, and we pick the top k nodes that have the highest betweenness.

This algorithm performed better than thompson sampling in some cases, and worse than Thompsons sampling in other cases. So, for our node submissions for day2, we ran thompsons sampling vs combined strats to see which one won, and we submitted the seed nodes that did the best. Below is the pseudocode for this:

Algorithm 1 Combine Thompson Sampling and Max Degree

```

1: Initialize an empty list for important nodes: imp_nodes
2:                                     ▷ Thompson Sampling
3: Obtain nodes using Thompson Sampling strategy
4: Convert nodes to integers
5: Combine Thompson Sampling nodes with Max Degree nodes, ensuring
   uniqueness
6:                                     ▷ Cluster Analysis
7: Identify and sort clusters in the graph by size
8: Select the four largest clusters
9:
10: for each combined node do
11:   Check if the node is in any of the largest clusters
12:   if yes then
13:     Add the node to the list of important nodes: imp_nodes
14:   end if
15: end for
16:                                     ▷ Sort important nodes by betweenness centrality
17: Sort the list of important nodes by their betweenness centrality values in
   descending order
18:                                     ▷ Fallback Handling
19: if the list of important nodes is empty then
20:   Use all combined nodes for selection
21: end if
22:                                     ▷ Select top nodes
23: Select the top  $k$  nodes from the sorted list of important nodes
24: return The selected nodes

```

6 Strategy 5 - Communities

Our next strategy utilized the combination of communities and highest degree nodes because we realized we wanted to position ourselves in highly clustered areas since we learned in class that the higher the clustering, the harder it will be to convert your node to the other team's node. We saw in the competition, we were not doing a good job of retaining our nodes. Thus, we first split the graph into communities using a `networkx` function. Communities are defined as a

collection of nodes that are densely connected internally but are less connected outside the collection. This is exactly what we want, since each node would be connected to more nodes of our color rather than nodes of another color, so it would prevent other teams from taking over. This technique would only work if the graph was connected. Then, we sorted the communities based on highest average degree. We did this since we wanted to account for max degree as well as centrality. Max degree is important since it would account for how quickly we spread in the graph. If our node connects to more nodes, our color will spread faster. Once we sorted the communities, we retrieved the k highest average degree communities. The k was determined experimentally, so it was chosen by running the program on a series of graphs with different k values. Then, for each of the k communities, we selected the highest degree nodes (for the reasons described earlier), shuffled the final list, and retrieved the required number of starting nodes. We predicted that this may do well since it not only holds onto already colored nodes well (by selecting highly clustered region), but also it spreads through the graph faster (by selecting highest degree nodes in the highly clustered regions).

7 Strategy 6 - final strategy - mix of different centrality measures

Our final strategy was to brute force different centrality methods and return the best one. We had a bag of measures. The first was degree centrality. We thought this may be important because it measures the connectedness of a node in the network, which is important for attempting to spread our color in the graph. The next was eigenvector centrality, which we thought is important because it measures the impact a node has not only on its direct neighbors, but also on its indirect neighbors. The more a node is connected to other impactful nodes, on average, the faster our team color would cascade. We also considered betweenness centrality because it identifies nodes that are intermediaries or bridges between connected components. It identifies the nodes that have a large influence on how information flows between pairs of nodes. In our case, this identifies nodes that can spread our team color most efficiently. Moreover, we considered pagerank because it is currently used in practice, especially in the Google search engine. It works by calculating a score based on a node's connections and based on the importance of the nodes it is connected to. Our theory was that the higher this score, the higher the chance of cascading since the nodes would be connected to important nodes which would be connected to many highly connected nodes. Lastly, we also had harmonic centrality because it not only considers well-connected nodes, but also nodes that have short distances to other nodes. Having short distance to other nodes means that we can capture a larger number of nodes early on, which helps spread our color throughout the graph.

Now, for the actual algorithm, we ran (ran the simulation) all of these

strategies against an arbitrary strategy, the degree centrality. If a strategy claimed more nodes than the degree centrality strategy and all the other strategies against degree centrality, then we chose nodes from that strategy. If there was no such strategy, then we chose the degree centrality strategy. We did this because it allowed us to pick the best strategy for each graph. Since all graphs are different, it allowed us to tailor our strategy for each graph. These measures allowed our color to retain captured nodes while also spreading at a fairly efficient rate.

8 Conclusion

During the competition, we ran these strategies against a generic max degree strategy, and we chose the strategy that performed the best. In general, it seemed like strategy 6 performed the best because it was able to find the best measure for each graph, whereas for some of the other strategies, the weights, k values, etc were hard-coded and were less general for a variety of graphs.

9 Improvements for next year

We put in a lot of effort into thinking about our machine learning approach to this problem. When we noticed that some centrality measures were beating our ML approaches, we should have experimented sooner with different centralities. But, since we tied with TA hard this year and were unable to beat TA hard, I would use my first strategy, and make the reward function just to beat our combined clustering algorithm. This would ensure that we beat our combined clustering algorithm every time, which always picked the same nodes as TA hard, and so we would then beat TA hard almost every time. Since a lot of teams were also losing to TA hard, beating TA hard would have also helped us perform better in the overall tournament.

10 Contributions of each team member

- Both of us wrote the report together
- Purvi - Wrote the reward function for strategy2 and strategy3, GraphProcessing code. Wrote Strategy5, experimented with another strategy not mentioned in file, but can be seen in our code.
- Riti - Wrote strategy1, strategy2, strategy3, strategy4, strategy6. Also experimented with other potential strategies not mentioned in the report, but can be seen in our code.

11 Link to code

github link: <https://github.com/pseghal2/Pandemaniac>

12 Citations

- <https://towardsdatascience.com/multi-armed-bandits-thompson-sampling-algorithm-fea205cf31df>
- https://github.com/kfoofw/bandit_simulations/blob/master/python/multiarmed_bandits/analysis/ucb.r
- <https://www.analyticsvidhya.com/blog/2018/09/reinforcement-multi-armed-bandit-scratch-python/>